

Walkthrough del código de grafico_circular_bienes_v1.Rmd

Este documento presenta un análisis detallado del código, explicando cada sección, su propósito y funcionamiento.

1. Configuración inicial (Metadatos YAML y setup)

```
---
output:
  word_document: default
  pdf_document:
    keep_tex: true
    extra_dependencies: ["graphicx", "float"]
  html_document: default
---
```

Esta sección define los formatos de salida soportados (Word, PDF y HTML) con configuraciones específicas para PDF que incluyen mantener el archivo TEX intermedio y añadir dependencias para manejo de gráficos y posicionamiento.

```
`r`{r setup, include=FALSE}
# Configuración para todos los formatos de salida
Sys.setlocale(category = "LC_NUMERIC", locale = "C")
options(OutDec = ".")
```

Estas líneas configuran el entorno R para usar punto como separador decimal, lo cual es crucial para la consistencia en diferentes configuraciones regionales.

```
# Configurar el motor LaTeX globalmente
options(tikzLatex = "pdflatex")
options(tikzXelatex = FALSE)
options(tikzLatexPackages = c(
  "\\usepackage{tikz}",
  "\\usepackage{colortbl}",
  "\\usepackage{graphicx}",
  "\\usepackage{float}"
))
```

Aquí se configura el motor LaTeX para la generación de PDF, especificando pdflatex como compilador y cargando paquetes necesarios para gráficos y tablas.

```
library(exams)
library(reticulate)
library(digest)
library(testthat)
library(knitr)
```

Carga las bibliotecas necesarias: - **exams**: Para la generación de ejercicios - **reticulate**: Para la integración con Python - **digest**: Para funciones hash (usado en pruebas) - **testthat**: Para pruebas unitarias - **knitr**: Para la generación de documentos

```
typ <- match_exams_device()
options(scipen = 999)
knitr::opts_chunk$set(
  warning = FALSE,
  message = FALSE,
  fig.showtext = FALSE,
```

```
fig.cap = "",
fig.keep = 'all',
dev = c("png", "pdf"),
dpi = 150,
fig.pos = "H"
)
```

Esta sección configura opciones globales para los chunks de código:

- Desactiva notación científica
- Suprime advertencias y mensajes
- Configura la generación de figuras en formatos PNG y PDF
- Establece la resolución de las imágenes a 150 DPI
- Fija la posición de las figuras como “H” (here) para LaTeX

```
# Configuración para chunks de Python
knitr::knit_engines$set(python = function(options) {
  knitr::engine_output(options, options$code, '')
})

# Asegurar que Python esté correctamente configurado
use_python(Sys.which("python"), required = TRUE)
```

Configura el motor de Python para reticulate y asegura que Python esté disponible en el sistema.

2. Definición y aleatorización de variables

```
```{r DefinicionDeVariables, message=FALSE, warning=FALSE, results='asis'}
options(OutDec = ".") # Asegurar punto decimal en este chunk

Establecer semilla aleatoria
set.seed(sample(1:10000, 1))
```

Inicia la sección de definición de variables, asegurando el uso del punto decimal y estableciendo una semilla aleatoria para garantizar reproducibilidad pero con variación entre ejecuciones.

```
Aleatorización del contexto del problema
contextos <- c(
 "empresa", "organización", "compañía", "institución", "corporación",
 "entidad", "firma", "negocio", "sociedad", "grupo empresarial"
)
contexto <- sample(contextos, 1)
```

Define y aleatoriza el contexto organizacional del problema.

```
Aleatorización de los tipos de bienes
tipos_bien1 <- c("carro", "auto", "vehículo", "automóvil", "coche")
tipos_bien2 <- c("casa", "vivienda", "residencia", "hogar", "domicilio")
tipos_bien3 <- c("apartamento", "departamento", "piso", "condominio")

bien1 <- sample(tipos_bien1, 1)
bien2 <- sample(tipos_bien2, 1)
bien3 <- sample(tipos_bien3, 1)
```

Define y aleatoriza los tipos de bienes que se utilizarán en el problema.

```
Aleatorización de términos para el enunciado
terminos_encuesta <- c("encuesta", "sondeo", "estudio", "investigación", "consulta")
termino_encuesta <- sample(terminos_encuesta, 1)

terminos_personas <- c("personas", "empleados", "trabajadores", "colaboradores", "miembros")
termino_personas <- sample(terminos_personas, 1)

terminos_bienes <- c("bienes", "posesiones", "propiedades", "activos", "patrimonios")
termino_bienes <- sample(terminos_bienes, 1)

terminos_resultados <- c("resultados", "datos", "información", "estadísticas", "cifras")
termino_resultados <- sample(terminos_resultados, 1)
```

Aleatoriza términos adicionales para enriquecer el enunciado del problema.

```
Aleatorización de colores para el gráfico circular (paletas oscuras)
paleta_colores <- list(
 c("#C62828", "#2E7D32", "#1565C0", "#EF6C00", "#6A1B9A"), # Paleta oscura 1
 c("#D32F2F", "#388E3C", "#1976D2", "#F57C00", "#7B1FA2"), # Paleta saturada
 c("#B71C1C", "#1B5E20", "#0D47A1", "#E65100", "#4A148C"), # Paleta oscura 2
 c("#AD1457", "#00695C", "#283593", "#FF6F00", "#4A148C"), # Paleta oscura 3
 c("#880E4F", "#1B5E20", "#0D47A1", "#BF360C", "#4A148C") # Paleta oscura 4
)
paleta_seleccionada <- sample(paleta_colores, 1)[[1]]
```

Define y selecciona aleatoriamente una paleta de colores para el gráfico circular.

```
Aleatorización de los porcentajes manteniendo coherencia matemática
Generación de porcentajes iniciales para cada categoría
set_porcentajes <- function() {
 repeat {
 # Generar valores base aleatorios
 p_bien1_bien3 <- sample(15:35, 1) # Porcentaje de bien1 y bien3
 p_solo_bien3 <- sample(15:30, 1) # Porcentaje solo bien3
 p_solo_bien2 <- sample(25:40, 1) # Porcentaje solo bien2
 p_solo_bien1 <- sample(5:10, 1) # Porcentaje solo bien1

 # Calcular el porcentaje restante para bien1 y bien2
 p_bien1_bien2 <- 100 - (p_bien1_bien3 + p_solo_bien3 + p_solo_bien2 + p_solo_bien1)

 # Validar que el porcentaje restante sea positivo y razonable
 if (p_bien1_bien2 >= 10 && p_bien1_bien2 <= 25) {
 return(c(p_bien1_bien3, p_solo_bien3, p_solo_bien2, p_solo_bien1, p_bien1_bien2))
 }
 }
}
```

Define una función para generar porcentajes aleatorios que sumen exactamente 100% y estén dentro de rangos razonables.

```
Obtener porcentajes válidos
porcentajes <- set_porcentajes()

p_bien1_bien3 <- porcentajes[1] # Porcentaje de bien1 y bien3
p_solo_bien3 <- porcentajes[2] # Porcentaje solo bien3
p_solo_bien2 <- porcentajes[3] # Porcentaje solo bien2
```

```

p_solo_bien1 <- porcentajes[4] # Porcentaje solo bien1
p_bien1_bien2 <- porcentajes[5] # Porcentaje de bien1 y bien2

Verificar que suman 100%
test_that("Los porcentajes suman 100%", {
 expect_equal(sum(porcentajes), 100)
})

```

Genera los porcentajes y verifica que sumen exactamente 100%.

```

Valor conocido para el cálculo: Número de personas con bien1 y bien3
personas_bien1_bien3 <- sample(c(72, 96, 120, 144, 168, 192, 216, 240), 1)

Calcular el total de personas
total_personas <- round(personas_bien1_bien3 * 100 / p_bien1_bien3)

Calcular el número de personas en cada categoría
personas_solo_bien3 <- round(total_personas * p_solo_bien3 / 100)
personas_solo_bien2 <- round(total_personas * p_solo_bien2 / 100)
personas_solo_bien1 <- round(total_personas * p_solo_bien1 / 100)
personas_bien1_bien2 <- round(total_personas * p_bien1_bien2 / 100)

Recalcular personas_bien1_bien3 para asegurar que el total sea correcto
personas_bien1_bien3 <- total_personas - (personas_solo_bien3 + personas_solo_bien2 + personas_solo_bien1)

```

Selecciona aleatoriamente un valor conocido (personas con bien1 y bien3) y calcula el resto de valores manteniendo la coherencia matemática.

```

Asegurar que todos los valores son positivos y razonables
personas <- c(personas_bien1_bien3, personas_solo_bien3, personas_solo_bien2, personas_solo_bien1, personas_bien1_bien2)
test_that("Todas las categorías tienen al menos una persona", {
 expect_true(all(personas > 0))
})

Asegurar que la suma de todas las categorías es igual al total
test_that("La suma de todas las categorías es igual al total", {
 expect_equal(sum(personas), total_personas)
})

```

Verifica que todos los valores calculados sean positivos y que la suma sea correcta.

```

Recalcular los porcentajes reales basados en los números de personas (para mantener coherencia)
p_bien1_bien3 <- round(personas_bien1_bien3 / total_personas * 100)
p_solo_bien3 <- round(personas_solo_bien3 / total_personas * 100)
p_solo_bien2 <- round(personas_solo_bien2 / total_personas * 100)
p_solo_bien1 <- round(personas_solo_bien1 / total_personas * 100)
p_bien1_bien2 <- round(personas_bien1_bien2 / total_personas * 100)

Ajustar para asegurar que suman 100%
total_porcentaje <- p_bien1_bien3 + p_solo_bien3 + p_solo_bien2 + p_solo_bien1 + p_bien1_bien2
if (total_porcentaje > 100) {
 # Si suman más de 100, restar la diferencia de la categoría más grande
 max_idx <- which.max(porcentajes)
 porcentajes[max_idx] <- porcentajes[max_idx] - (total_porcentaje - 100)
} else if (total_porcentaje < 100) {
 # Si suman menos de 100, añadir la diferencia a la categoría más pequeña
 min_idx <- which.min(porcentajes)
 porcentajes[min_idx] <- porcentajes[min_idx] + (100 - total_porcentaje)
}

```

```

min_idx <- which.min(porcentajes)
porcentajes[min_idx] <- porcentajes[min_idx] + (100 - total_porcentaje)
}

```

Recalcula los porcentajes basados en los números de personas y ajusta para asegurar que sumen exactamente 100%.

```

p_bien1_bien3 <- porcentajes[1]
p_solo_bien3 <- porcentajes[2]
p_solo_bien2 <- porcentajes[3]
p_solo_bien1 <- porcentajes[4]
p_bien1_bien2 <- porcentajes[5]

Asegurar que los porcentajes suman exactamente 100%
test_that("Los porcentajes ajustados suman 100%", {
 expect_equal(sum(porcentajes), 100)
})

```

Actualiza los porcentajes y verifica nuevamente que sumen 100%.

```

Determinar la respuesta correcta y generar tres distractores plausibles
La respuesta correcta es el número de personas con solo bien3
respuesta_correcta <- personas_solo_bien3

Generar distractores plausibles
distractor1 <- round(personas_bien1_bien3 / 4) # Un cuarto del valor dado en el problema
distractor2 <- round(total_personas * p_bien1_bien3 / 100) # Confundir entre porcentaje y valor absoluto
distractor3 <- personas_bien1_bien3 # El mismo valor dado en el problema

Asegurarse de que todos los distractores son diferentes de la respuesta correcta
if (distractor1 == respuesta_correcta) distractor1 <- distractor1 + sample(5:15, 1)
if (distractor2 == respuesta_correcta) distractor2 <- distractor2 - sample(5:15, 1)
if (distractor3 == respuesta_correcta) distractor3 <- distractor3 + sample(5:15, 1)

Asegurarse de que todos los distractores son diferentes entre sí
while (length(unique(c(distractor1, distractor2, distractor3))) < 3) {
 if (distractor1 == distractor2) distractor1 <- distractor1 + sample(5:10, 1)
 if (distractor2 == distractor3) distractor2 <- distractor2 - sample(5:10, 1)
 if (distractor1 == distractor3) distractor3 <- distractor3 + sample(5:10, 1)
}

```

Define la respuesta correcta y genera distractores plausibles basados en errores comunes, asegurando que sean diferentes entre sí y de la respuesta correcta.

```

Crear un vector con todas las opciones y mezclarlas
opciones <- c(respuesta_correcta, distractor1, distractor2, distractor3)
names(opciones) <- c("correcta", "distractor1", "distractor2", "distractor3")
opciones_mezcladas <- sample(opciones)

Identificar la posición de la respuesta correcta en las opciones mezcladas
indice_correcto <- which(opciones_mezcladas == respuesta_correcta)

Crear el vector de solución para r-exams
solucion <- rep(0, 4)
solucion[indice_correcto] <- 1

```

Mezcla las opciones de respuesta y crea el vector de solución para r-exams.

### 3. Generación del gráfico circular

```
```{r generar_grafico_circular, message=FALSE, warning=FALSE}
options(OutDec = ".") # Asegurar punto decimal en este chunk

# Código Python para generar el gráfico circular
codigo_python <- paste0("
import matplotlib
matplotlib.use('Agg') # Usar backend no interactivo
import matplotlib.pyplot as plt
import numpy as np

# Datos para el gráfico
labels = [' ', bien1, " y ", bien3, "", 'Solo ', bien3, "", 'Solo ', bien2, "",
          'Solo ', bien1, "", ' ', bien1, " y ", bien2, "']
sizes = [ , p_bien1_bien3, " ", " ", p_solo_bien3, " ", " ", p_solo_bien2, " ",
          " ", p_solo_bien1, " ", " ", p_bien1_bien2, " ]
colors = [' ', paleta_seleccionada[1], "", "", paleta_seleccionada[2], "",
          "", paleta_seleccionada[3], "", "", paleta_seleccionada[4], "",
          "", paleta_seleccionada[5], "']

```

Inicia la generación del gráfico circular con Python, definiendo las etiquetas, tamaños y colores.

```
# Explode para destacar ligeramente el sector con bien1 y bien3
explode = (0.03, 0, 0, 0, 0, 0)

# Crear figura con más espacio para acomodar las etiquetas externas
plt.figure(figsize=(7, 6)) # Aumentar tamaño para dar espacio a las etiquetas

# Crear gráfico circular con bordes blancos
wedges, texts, autotexts = plt.pie(
    sizes,
    explode=explode,
    colors=colors,
    shadow=True,
    startangle=45, # Cambiar ángulo para mejor distribución
    autopct='%d%%',
    pctdistance=0.75, # Ubicar porcentajes más cerca del borde exterior
    wedgeprops={'edgecolor': 'white', 'linewidth': 1},
    textprops={'fontsize': 10, 'fontweight': 'bold', 'color': 'white'})

```

Crea el gráfico circular con configuraciones detalladas para mejorar la visualización.

```
# Configuración estética de los textos de porcentaje con recuadros
for autotext in autotexts:
    autotext.set_fontsize(9) # Tamaño de fuente para porcentajes
    autotext.set_weight('bold')
    # Crear un recuadro semitransparente para los porcentajes
    bbox_props = {'boxstyle': 'round,pad=0.2',
                  'facecolor': 'black',
                  'alpha': 0.7,
                  'edgecolor': 'none'}

```

```

autotext.set_bbox(bbox_props)

# Eliminar los textos del pie (los reemplazaremos con etiquetas externas)
for text in texts:
    text.set_visible(False)

```

Configura los textos de porcentaje con recuadros y elimina las etiquetas predeterminadas para reemplazarlas con etiquetas externas.

```

# Crear etiquetas externas con líneas conectoras
bbox_props = {'boxstyle': 'round,pad=0.3',
              'facecolor': 'white',
              'edgecolor': 'gray',
              'alpha': 0.9}

# Calcular posiciones de etiquetas externas optimizadas para evitar solapamiento
def get_label_position(angle_rad, wedge_size):
    # Ajustar distancia basada en el tamaño del sector
    # Sectores más pequeños tienen etiquetas más alejadas para evitar solapamiento
    distance_factor = 1.25 if wedge_size < 15 else 1.15

    # Determinar coordenadas
    x = distance_factor * np.cos(angle_rad)
    y = distance_factor * np.sin(angle_rad)

    # Ajustar alineación basada en cuadrante
    if x < 0:
        ha = 'right'
    else:
        ha = 'left'

    if y < 0:
        va = 'top'
    else:
        va = 'bottom'

    # Para sectores muy pequeños, ajustar más la posición para evitar solapamiento
    if wedge_size < 10:
        x *= 1.1
        y *= 1.1

    return x, y, ha, va

```

Define una función para calcular posiciones óptimas para las etiquetas externas, evitando solapamientos.

```

# Colocar etiquetas externas con líneas conectoras
for i, wedge in enumerate(wedges):
    # Calcular ángulo medio del sector
    ang = (wedge.theta1 + wedge.theta2) / 2
    ang_rad = np.radians(ang)

    # Calcular coordenadas para el inicio de la línea conectora (en el borde del sector)
    # El factor 0.85 asegura que la línea empiece cerca del borde del sector
    x_edge = 0.85 * np.cos(ang_rad)
    y_edge = 0.85 * np.sin(ang_rad)

```

```

# Obtener posición optimizada para la etiqueta
x_label, y_label, ha, va = get_label_position(ang_rad, sizes[i])

# Dibujar línea conectora
con = plt.annotate('',
                    xy=(x_edge, y_edge), # Inicio (en el borde del sector)
                    xytext=(x_label * 0.95, y_label * 0.95), # Fin (cerca de la etiqueta)
                    arrowprops=dict(arrowstyle='-', color='gray', lw=0.8))

# Añadir etiqueta con recuadro blanco
plt.text(x_label, y_label, labels[i],
         fontsize=8,
         fontweight='bold',
         ha=ha,
         va=va,
         bbox=bbox_props,
         zorder=10)

```

Coloca las etiquetas externas con líneas conectoras a los sectores del gráfico.

```

# Asegurar que el gráfico sea un círculo
plt.axis('equal')

# Añadir título en la parte inferior del gráfico
plt.figtext(0.5, 0.01, # Posición x=0.5 (centro), y=0.01 (parte inferior)
            'Distribución de bienes',
            fontsize=12,
            fontweight='bold',
            color='#333333',
            ha='center') # Alineación horizontal centrada

# Ajustar los márgenes para dejar espacio a las etiquetas externas
plt.tight_layout(pad=1.5, rect=[0, 0.05, 1, 0.95]) # Ajustar rect para dar más espacio

# Guardar en múltiples formatos para asegurar compatibilidad
plt.savefig('grafico_circular.png', dpi=150, bbox_inches='tight',
            transparent=True, format='png')
plt.savefig('grafico_circular.pdf', dpi=150, bbox_inches='tight',
            transparent=True, format='pdf')
plt.close()
")

# Ejecutar código Python para generar la figura
py_run_string(codigo_python)

```

Finaliza la configuración del gráfico, añade un título, ajusta los márgenes y guarda la figura en formatos PNG y PDF.

4. Pregunta y opciones de respuesta

Question
=====

Se realizó un(a) `r termino\_encuesta` a un grupo de `r termino\_personas` de un(a) `r contexto` sobre el


```

`{r mostrar_grafico_circular, echo=FALSE, results='asis', fig.align="center"}
# Detectar si se está generando para Moodle
# Formatos para Moodle y similares
formatos_moodle <- c("exams2moodle", "exams2qti12",
                     "exams2qti21", "exams2openolat")
es_moodle <- (match_exams_call() %in% formatos_moodle)

# Mostrar la imagen del gráfico circular
if (es_moodle) {
  # Tamaño para Moodle
  cat("![grafico_circular.png]{width=60%}")
} else {
  # Tamaño para PDF/Word
  cat("![grafico_circular.png]{width=70%}")
}

```

Si r personas bien1 bien3 r termino_personas de el(la) r contexto poseen r bien1 y r bien3, ¿cuántas personas poseen solo r bien3?

Answerlist

- r opciones_mezcladas[1]
- r opciones_mezcladas[2]
- r opciones_mezcladas[3]
- r opciones_mezcladas[4]

Define la pregunta, muestra el gráfico circular (con tamaño adaptado según el formato de salida) y presenta las opciones de respuesta.

Aspectos destacables de esta sección:

1. **Adaptabilidad al formato de salida**: El código detecta si se está generando para Moodle u otro fo
2. **Uso de variables aleatorizadas**: El enunciado utiliza las variables aleatorizadas previamente def
3. **Estructura clara**: La pregunta está estructurada de manera clara, proporcionando primero el conte
4. **Opciones de respuesta mezcladas**: Las opciones de respuesta se presentan en orden aleatorio, con

5. Solución

```

``r
Solution
=====

```

Para resolver este problema, necesitamos aplicar proporciones y regla de tres a partir de la información

Paso 1: Identificar los datos conocidos

- * Sabemos que r personas bien1 bien3 r termino_personas poseen r bien1 y r bien3.
- * Según el gráfico circular, este grupo representa el r p_bien1_bien3% del total.
- * También observamos en el gráfico que las personas que poseen solo r bien3 representan el r p_solo_l

Paso 2: Calcular el número total de personas

Para encontrar el total de `r termino\_personas` en la `r contexto`, utilizamos la siguiente relación de

Si `r p\_bien1\_bien3`% del total = `r personas\_bien1\_bien3` `r termino\_personas`
Entonces 100% del total = X `r termino\_personas`

Aplicando regla de tres:

```
* X = (`r personas_bien1_bien3` × 100%) ÷ `r p_bien1_bien3`%  
* X = `r personas_bien1_bien3` × 100 ÷ `r p_bien1_bien3`  
* X = `r total_personas` `r termino_personas`
```

Por lo tanto, el total de `r termino\_personas` en la `r contexto` es `r total\_personas`.

Paso 3: Calcular el número de personas que poseen solo `r bien3`

Una vez conocido el total, podemos calcular cuántas personas poseen solo `r bien3` utilizando el porcen

Si 100% del total = `r total\_personas` `r termino\_personas`
Entonces `r p\_solo\_bien3`% del total = Y `r termino\_personas`

Aplicando regla de tres:

```
* Y = (`r p_solo_bien3`% × `r total_personas`) ÷ 100%  
* Y = (`r p_solo_bien3` × `r total_personas`) ÷ 100  
* Y = `r p_solo_bien3` × total_personas / 100`  
* Y = `r respuesta_correcta` `r termino_personas`
```

Paso 4: Verificación de la respuesta

Podemos verificar nuestra respuesta comprobando que los números calculados son coherentes con los porcen

```
* `r bien1` y `r bien3`: `r personas_bien1_bien3` `r termino_personas` (`r p_bien1_bien3`% del total)  
* Solo `r bien3`: `r respuesta_correcta` `r termino_personas` (`r p_solo_bien3`% del total)  
* Solo `r bien2`: `r personas_solo_bien2` `r termino_personas` (`r p_solo_bien2`% del total)  
* Solo `r bien1`: `r personas_solo_bien1` `r termino_personas` (`r p_solo_bien1`% del total)  
* `r bien1` y `r bien2`: `r personas_bien1_bien2` `r termino_personas` (`r p_bien1_bien2`% del total)
```

La suma de todas estas categorías es `r sum(c(personas_bien1_bien3, personas_solo_bien3, personas_solo_

Conclusión

Por lo tanto, `r respuesta\_correcta` `r termino\_personas` de la `r contexto` poseen solo `r bien3`.

Answerlist

```
- `r if(solucion[1] == 1) "Verdadero" else "Falso"`  
- `r if(solucion[2] == 1) "Verdadero" else "Falso"`  
- `r if(solucion[3] == 1) "Verdadero" else "Falso"`  
- `r if(solucion[4] == 1) "Verdadero" else "Falso"`
```

Proporciona una solución detallada paso a paso, explicando el razonamiento matemático y verificando la respuesta.

Aspectos destacables de esta sección:

1. **Estructura pedagógica:** La solución está organizada en pasos claros y secuenciales, facilitando la comprensión del proceso de resolución.
2. **Explicación detallada:** Cada paso incluye una explicación detallada del razonamiento matemático aplicado.

3. **Uso de variables aleatorizadas:** La solución utiliza las variables aleatorizadas, lo que permite que sea coherente con la versión específica del problema generado.
4. **Verificación de la respuesta:** Incluye un paso de verificación que demuestra la coherencia de la solución con los datos del problema.
5. **Formato matemático claro:** Presenta las operaciones matemáticas de manera clara y estructurada, facilitando el seguimiento del proceso de cálculo.

6. Metainformación

```
Meta-information
=====
exname: proporciones_diagrama_circular
extype: schoice
exsolution: `r paste(as.integer(solucion), collapse="")`
exshuffle: TRUE
exsection: Estadística|Proporciones|Interpretación de gráficos
```

Define la metainformación para r-exams: - **exname:** Nombre del ejercicio - **extype:** Tipo de ejercicio (schoice = selección única) - **exsolution:** Vector de solución (1 para la respuesta correcta, 0 para las incorrectas) - **exshuffle:** Indica si las opciones deben mezclarse - **exsection:** Categorización del ejercicio

Aspectos destacables del código

1. **Aleatorización robusta:** El código implementa múltiples capas de aleatorización (términos, valores, colores) para generar numerosas variantes del mismo problema.
2. **Coherencia matemática:** Se realizan múltiples verificaciones para asegurar que los cálculos sean matemáticamente coherentes.
3. **Visualización avanzada:** El gráfico circular incluye características avanzadas como etiquetas externas con líneas conectoras, recuadros para los porcentajes y optimización para evitar solapamientos.
4. **Adaptabilidad a diferentes formatos:** El código detecta el formato de salida y ajusta la presentación en consecuencia.
5. **Solución pedagógica:** La solución está estructurada de manera didáctica, explicando paso a paso el razonamiento matemático.
6. **Distractores plausibles:** Los distractores se generan basados en errores comunes, haciéndolos plausibles pero inequívocamente incorrectos.
7. **Pruebas integradas:** El código incluye pruebas unitarias integradas para verificar la coherencia de los datos generados.

Este ejercicio es un excelente ejemplo de cómo utilizar r-exams para crear problemas matemáticos interactivos con alta variabilidad y calidad pedagógica. La combinación de R y Python permite aprovechar las fortalezas de ambos lenguajes: R para la lógica del ejercicio y Python para la visualización avanzada.